

Cel: Nabycie umiejętności skomunikowania komputera z urządzeniem wyposażonym w mikrokontroler. Zadanie obejmuje naukę wykorzystania klasy CDC interfejsu USB oraz opracowanie prostego protokołu komunikacyjnego, umożliwiającego dwustronną komunikację operatora z urządzeniem za pośrednictwem terminala.

Zadanie 3 : Korzystając z bibliotek STM32F3 HAL oraz zestawu STM32F3DISCOVERY zrealizować program obsługujący dwukierunkową komunikację z komputerem zgodnie z poniższymi założeniami:

- 1) Program ma umożliwiać zmianę nastaw i odczyt wyników przez sprzętowy port USB mikrokontrolera. Komunikacja odbywa się z wykorzystaniem klasy USB CDC.
- 2) Wysłane przez terminal operatora ciągi znaków są interpretowane przez program mikrokontrolera. Ciągi są zorganizowane w ramki komunikacyjne i zawierają następujące pola:
 - a. Znak początku (np. znak specjalny z palety ASCII)
 - b. Znak polecenia (umożliwia wybór funkcji realizowanej przez mikrokontroler)
 - c. Pole danych (zakres wartości zależny od rodzaju funkcji)
 - d. Znak końca (np. znak specjalny z palety ASCII)
- 3) Odebrane polecenia są wykonywane przez program mikrokontrolera tylko jeśli znaki początku i końca są prawidłowe, polecenie znajduje się na liście obsługiwanych poleceń oraz pole danych jest zgodne z zakresem przewidzianym dla danej funkcji. Błędy w komunikacji sygnalizowane są podwójnym „mignięciem” diody PE9 (kolor czerwony), pozostałe diody nie zmieniają swoich stanów w procedurze sygnalizacji błędu. Sygnalizacja błędu ma być zrealizowana w postaci funkcji. Program sygnalizuje swoją gotowość cykliczną zmianą stanu diody PE8 (niebieska), która jest obsługiwana w za pomocą wybranego licznika (przerwanie licznika lub kanał PWM). [2 pkt]
- 4) Program mikrokontrolera ma obsługiwać następujące typy poleceń (lista dostępnych funkcji):
 - a. *Sterowanie stanem pojedynczej wybranej diody (z zakresu PE10-PE15)* [2 pkt]
Funkcja prezentuje możliwość zdalnej zmiany stanu cyfrowego wyjścia mikrokontrolera.
Parametr (pole danych) może przyjąć wartości dziesiętne (zapisane jako znaki ASCII) z listy:
0: zmiana stanu diody
1: dioda aktywna
2: dioda nieaktywna
Zrealizowanie polecenia ma być potwierdzane wysłaniem ustalonego ciągu znaków na terminal operatora (np. „ACK”)
 - b. *Nastawa intensywności świecenia diody (jedna z niewykorzystanych wcześniej diod obsługująca sprzętowy kanał PWM licznika)* [2pkt]
Funkcja prezentuje możliwość zdalnej zmiany parametru liczbowego.
Program ma umożliwiać zdalną nastawę intensywności świecenia poprzez zmianę wypełnienia sygnału PWM sterującego stanem diody. Parametr (pole danych) może przyjąć wartość dziesiętną z zakresu 0 – 9 (zapisane jako znaki ASCII). Zrealizowanie polecenia ma być potwierdzane wysłaniem ustalonego ciągu znaków na terminal operatora (np. „ACK”).
 - c. *Odczyt wartości licznika aktywacji przycisku USER* [2pkt]
Funkcja prezentuje możliwość zdalnego odczytu wartości liczbowej przechowywanej w mikrokontrolerze.
Parametr (pole danych) ma mieć wartość dziesiętną 0 (zapisaną jako znak ASCII).

Po odebraniu polecenia program ma wysyłać informację o liczbie wciśnień przycisku USER od ostatniego zresetowania mikrokontrolera. Odpowiedź ma być wysłana w formacie: „LICZBA WCIŚNIĘĆ: %d”, gdzie %d to liczba dziesiętna zapisana z wykorzystaniem znaków ASCII. Liczba wciśnień przycisku ma być zliczana poprzez użycie przerwania (detekcja zmiany stanu na linii PA0).

d. *Monitorowanie stanu przycisku USER [2pkt]*

Funkcja prezentuje możliwość ciągłego monitorowania stanu wejścia mikrokontrolera.

Po aktywacji funkcji mikrokontroler wysyła informacje o każdej zmianie stanu przycisku (np. „ON” / „OFF”), aktywność funkcji jest sygnalizowana włączeniem wybranej diody. Wciśnięcie przycisku ma być wykrywane poprzez użycie przerwania. Funkcja przestaje być aktywna po odebraniu przez mikrokontroler nowego polecenia lub po ponownym przesłaniu polecenia monitorowania stanu przycisku USER.

Wskazówki:

- a) Konfigurując mikrokontroler w CubeMX należy w sekcji Connectivity sprawdzić czy aktywny jest interfejs USB oraz w sekcji Middleware wybrać klasę Communication Device Class (CDC/Virtual COM Port)
- b) Konfigurując sygnały zegarowe należy ustawić 48 MHz jako częstotliwość taktowania dla interfejsu USB
- c) Do obsługi wirtualnego portu szeregowego na komputerze można skorzystać z dowolnego programu z funkcjonalnością terminala (np. Terminal 1.93b, Putty, YAT - Yet Another Terminal, RealTerm). Dla uproszczenia obsługi można korzystać z makr (jeśli terminal to umożliwia).
- d) Opis biblioteki USB dla STM32 (Middleware) znajduje się w instrukcji UM1734
- e) Założenia programu umożliwiają wydawanie poleceń o stałej długości ramki, co ułatwia analizowanie poleceń i wykrywanie błędów w komunikacji.
- f) Parametry liczbowe w polu danych mają być wysyłane i odbierane w postaci znaków alfanumerycznych z palety ASCII, konwersję znaków (cyfr) z tej palety na wartości liczbowe najprościej wykonać odejmując od wartości kodu znaku liczbę 48 (dziesiętnie)

Przydatne fragmenty kodu:

- a) Obsługa klasy CDC USB jest realizowana przez funkcje umieszczone w pliku:
USB_DEVICE/App/usbd_cdc_if.c
- b) Do wysyłania i odbioru ciągów znaków można użyć funkcji *CDC_Transmit_FS* oraz *CDC_Receive_FS*
- c) Dane do wysłania można przechowywać w buforze w postaci tablicy (np.: *uint8_t Cmd[30]*), a za pomocą funkcji *sprintf* można przygotować zawartość bufora przed wysłaniem. Przykładowe użycie (wymaga załączenia pliku nagłówkowego stdio.h lub string.h):

```
uint8_t Cmd[30];           //bufor w postaci tablicy
uint8_t CmdLength;         //liczba transmitowanych znaków
uint8_t CmdNum = 0;        //przykładowa zmienna liczbowo do wysłania
//przekazanie sformatowanego tekstu do bufora
CmdLength = sprintf(Cmd, "RUN_CMD_%d\r\n", CmdNum);
//wysłanie zawartości bufora
CDC_Transmit_FS (Cmd, CmdLength);
```
- d) Dostęp do zmiennych które zostały zdefiniowane w innym pliku, jest możliwy poprzez poprzedzenie nazwy i typu zmiennej słowem kluczowym *extern* (np. *extern uint8_t zmienna*)

- e) Przed przystąpieniem do analizy zawartości odebranego ciągu znaków zaleca się skopiowanie go do tablicy (bufora), która będzie dostępna wewnątrz funkcji analizującej przysłany ciąg. Nie zaleca się wykonywania czasochłonnych lub złożonych operacji wewnątrz uchwytu przerwania. Zawartość buforów można kopiować korzystając z funkcji `memcpy` lub `strncpy`. Wewnątrz funkcji odbioru danych z portu USB (`CDC_Receive_FS`) można to zrealizować np. poprzez polecenie:
- ```
strncpy(Cmd, Buf, *Len+1)
```
- gdzie: *Cmd* – wskaźnik na zewnętrzny bufor, do którego skopiowane zostaną odebrane dane,  
*Buf* – wskaźnik na tablicę z odebranymi danymi, parametr funkcji `CDC_Receive_FS`,  
*Len* – długość danych (liczba bajtów do skopiowania ), parametr funkcji `CDC_Receive_FS`